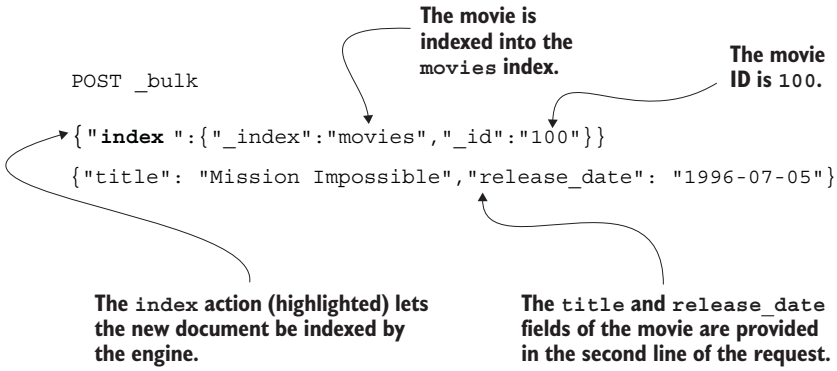


Figure 5.16 Indexing the new movie *Mission Impossible* using the `_bulk` API

If we execute this query, we have a document with ID 100 that is indexed into the `movies` document with the fields given in the second line. That is, the movie *Mission Impossible* is indexed into our store. The two lines make one request—we have to code two lines for each document we want to be acted on.

We can also shorten the metadata line. For example, we can remove the index and attach it to the URL, instead.

Listing 5.42 Bulk API with index embedded in the request URL

```
POST movies/_bulk
{"index":{"_id":"100"}}
{"title": "Mission Impossible","release_date": "1996-07-05"}
```

The URL includes the index.

The _index field has been removed.

We can also get rid of the `_id` field if we want system-generated random IDs for our movies. The following snippet demonstrates this approach.

Listing 5.43 Letting the system generate document IDs

```
POST movies/_bulk
{"index":{}}
{"title": "Mission Impossible","release_date": "1996-07-05"}
```

Both the _index and _id were removed

The system assigns the ID for this document, a random UUID.

Of course, you may wonder why we need to follow this bulk approach when we can index the same document using the document index API (`PUT movies/_doc/100`). That is a fair question, but we haven't yet unleashed the full power of the `_bulk` API. Suppose we have a requirement to index Tom Cruise's movies into our `movies` index. The request is written in the next listing.